INDIVIDUAL LABORATORY WORK

DevOps Fundamentals

Linux, Git, GitHub, Docker and Jenkins

Student information	Value
Name	Miras
Surname	Tusupkali
Group	IT2-2312
Student ID	38366
Course	DevOps
Verification date	18 October 2026

Project repository

github.com/Dezaster5/tusupkali-miras-devops

Execution environment

Ubuntu under WSL 2; Docker Engine 29.1.3; Docker Compose 2.40.3; Jenkins 2.580.1. The application uses the Python standard library. Docker Desktop is not required for this configured environment.

Verified result

Jenkins build #1 completed with SUCCESS. All five required stages passed, four HTTP integration tests passed, and the final application container reported healthy.

The screen captures and execution results in this report were obtained from this project. The continuous video with the student's own voice is a separate submission item.

1. Linux environment

The personalized Linux workspace is /home/dezaster_miras/devops/tusupkali_miras. It contains app, scripts, config, logs and backup directories, plus Tusupkali_Miras_info.txt, config/app.conf and logs/application.log.

Files, searching and permissions

Requirement	Implemented and demonstrated
Basic file operations	pwd, ls, cd, mkdir, touch, cp, mv, rm, cat, head and tail
Log processing	10 messages: 6 INFO, 2 WARNING and 2 ERROR; find, grep, grep -c and tail -n 5
File permissions	private.txt: 600 (rw-------); public.txt: 644 (rw-r--r--)
Processes	ps, ps aux and a top snapshot; PID, owner, CPU and memory usage
System script	Username, hostname, date, OS, disk/memory usage and information-file existence

Permission meaning: r = read, w = write, x = execute. The owner may edit both demonstration files; group and other users can only read public.txt. The private file is accessible only to its owner.

Actual execution excerpts

```
$ pwd
/home/dezaster_miras/devops/tusupkali_miras
$ grep -c ERROR logs/application.log
2
$ ./scripts/Tusupkali_Miras_system.sh
Name: Miras
Surname: Tusupkali
Group: IT2-2312
Student ID: 38366
Username: dezaster_miras
Student information file exists.
```

A process is an executing program, and its PID identifies that running instance. ps returns a snapshot; top refreshes resource usage. The system script and the full Linux demonstration both completed successfully in Ubuntu.

2. Git and GitHub

The repository has 12 real commits at verification, a main branch and a development branch. The development change added the /health endpoint and its readiness test, then was merged into main with an explicit merge commit. The repository is public under Dezaster5.

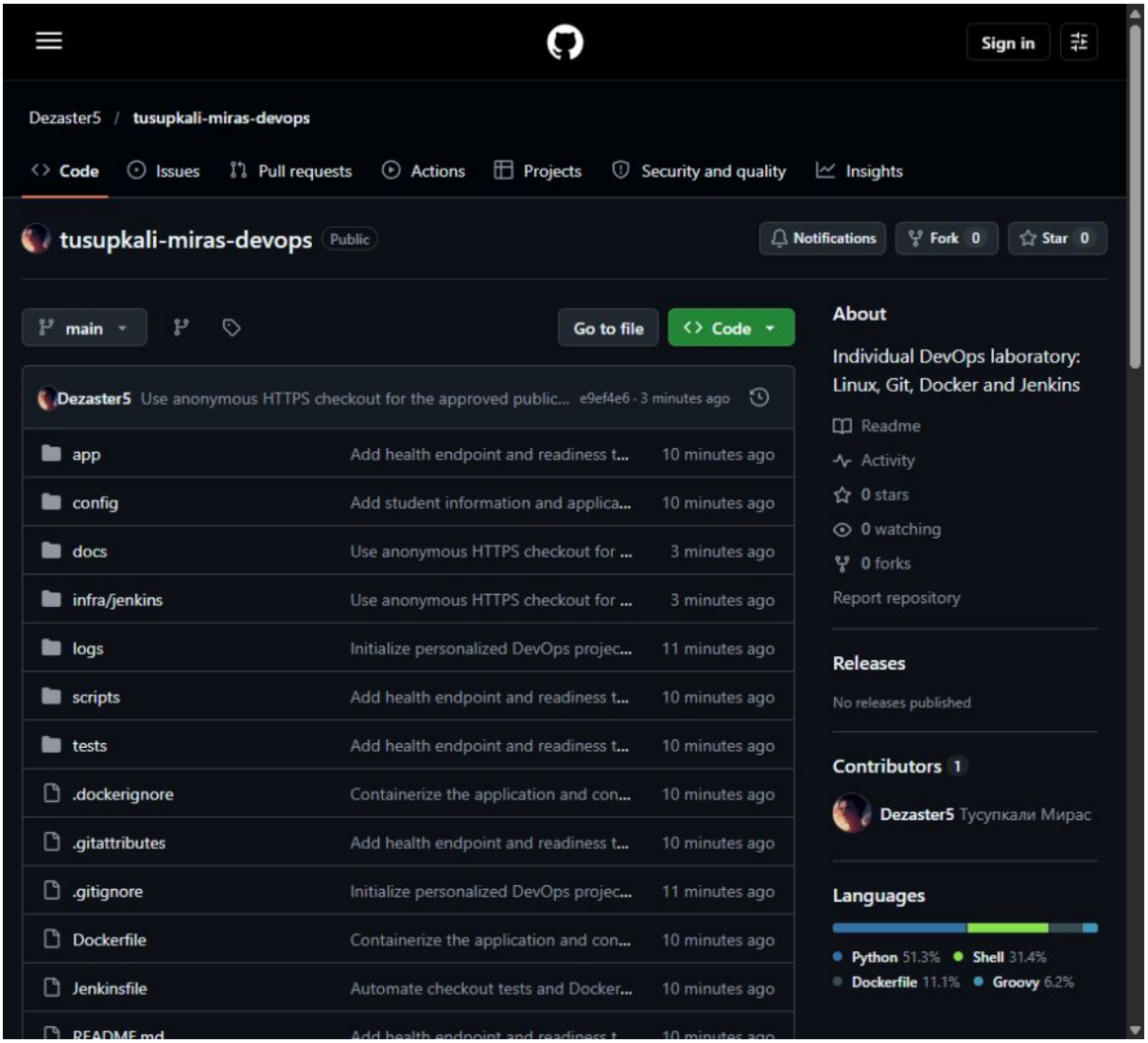


Figure 1. Actual public GitHub repository and personalized project files.

History and ignored files

```
e9db19f Document and simplify startup using the verified Ubuntu Docker engine
e9ef4e6 Use anonymous HTTPS checkout for the approved public repository
e9789fd Merge development health checks into main
7b30955 Add health endpoint and readiness test on development branch
9660cb9 Implement student application and HTTP integration tests
19c8620 Add student information and application configuration
dcaba4c Initialize personalized DevOps project structure
```

The .gitignore excludes *.log, backup/, .env and local Jenkins secrets. git check-ignore confirmed that the demonstration log and backup file are ignored. The Git author is configured for this repository using the student's GitHub noreply address. git push sends commits to GitHub; git pull obtains and integrates remote changes.

3. Docker application

The application returns student information at / and JSON at /api/student. The /health route is used to check readiness. The image name is tusupkali-miras-devops; the container name is tusupkali-miras-container.

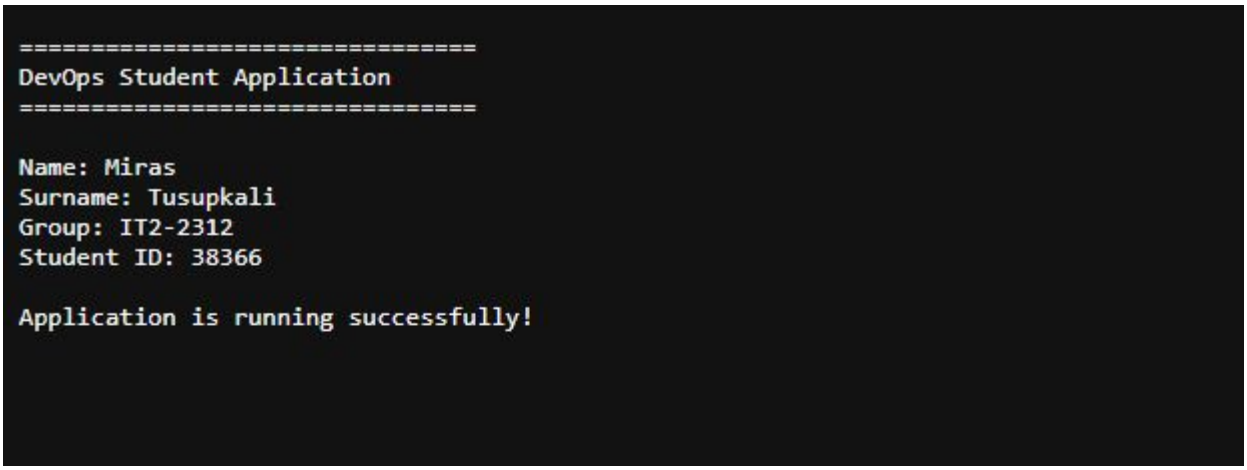


Figure 2. Detail of the actual application response at http://localhost:8000/.

Dockerfile instruction	Use in this project
FROM / WORKDIR	Python 3.12 slim base image; working directory /app
COPY / RUN	Copy source and configuration; create user and compile application
EXPOSE / CMD	Describe port 8000; run python -m app.main
USER / HEALTHCHECK	Run as UID 10001 (student); check /health

STUDENT_NAME, STUDENT_SURNAME, STUDENT_GROUP and STUDENT_ID are read from the environment. Tests verified that replacement values change the HTTP response. Credentials are not embedded in the application image. The container UID was verified as 10001.

Actual Docker verification

```
Image: tusupkali-miras-devops:1 / tusupkali-miras-devops:latest
Image ID: 1ffd4dac2ae0
Container: tusupkali-miras-container
Status after Pipeline: healthy
User: uid=10001(student) gid=10001(student)
Verified: build, run, ps, ps -a, logs, exec, stop, start, rm
Verified: docker compose up -d and docker compose down
```

An image is the reusable application template; a container is its runtime instance. Compose keeps the application's build, port and environment settings in one version-controlled file.

4. Jenkins automation

The Job Tusupkali_Miras_DevOps loads Jenkinsfile directly from the public GitHub repository over HTTPS. The local controller has authentication enabled and is available at http://localhost:8080/. Its configuration and Job are defined with JCasC and Job DSL.

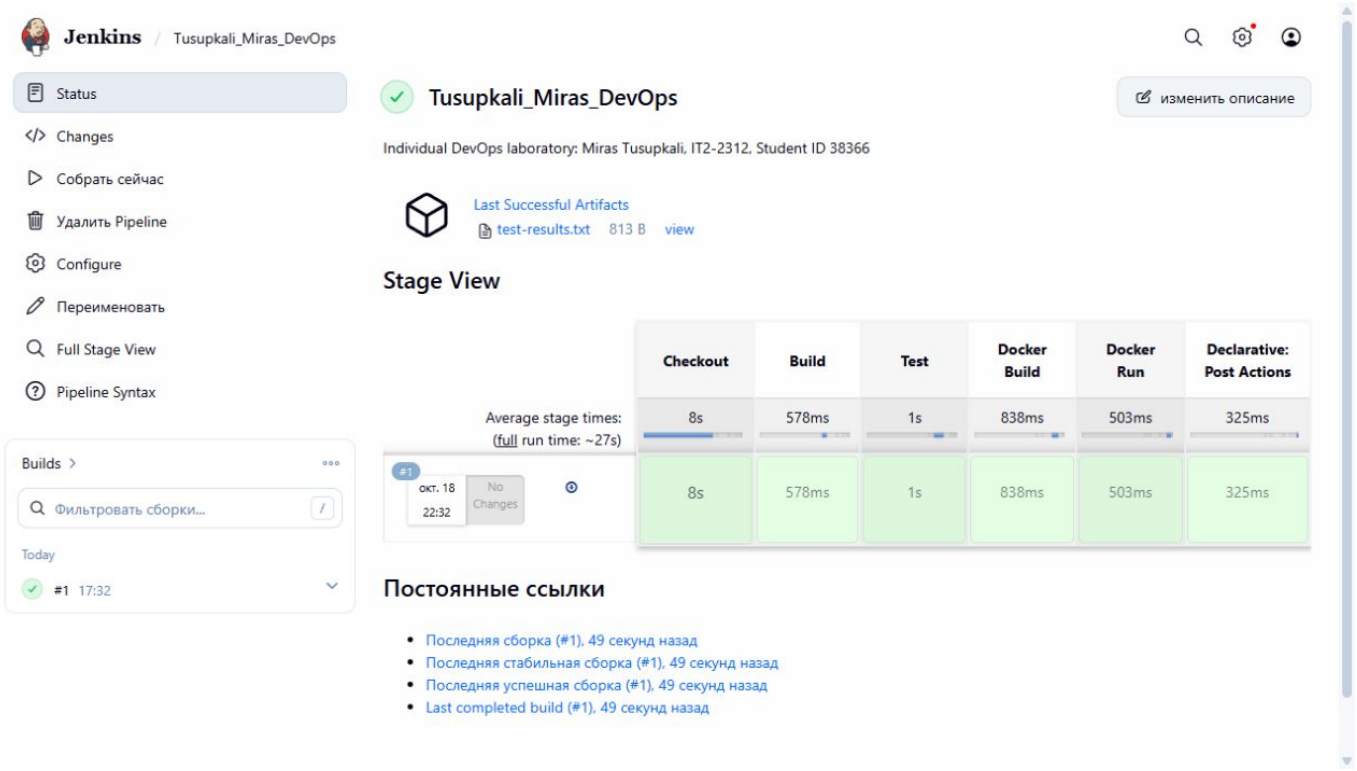


Figure 3. Actual Jenkins build #1 with successful stages.

Stage	Action	Result
Checkout	Get GitHub commit; print student information	SUCCESS
Build	Compile Python application and tests	SUCCESS
Test	Run four HTTP integration tests	SUCCESS
Docker Build	Build versioned image and latest tag	SUCCESS
Docker Run	Start container and verify /health	SUCCESS

Test coverage includes default student information, environment overrides, unknown-route HTTP 404 handling and readiness HTTP 200. Jenkins archives test-results.txt. The four STUDENT_* environment variables are configured in the Jenkinsfile and printed in Console Output.

This is a local trusted-project controller. Its web port is bound to localhost. The Docker socket allows the controller to build and run containers; it should not execute untrusted repositories.

5. Verified Pipeline result

Build #1 finished with SUCCESS in 27.7 seconds. The source revision checked out by Jenkins was:

```
e9db19f25f6c1b55274ec711cf5be2f33bdc1e63
```

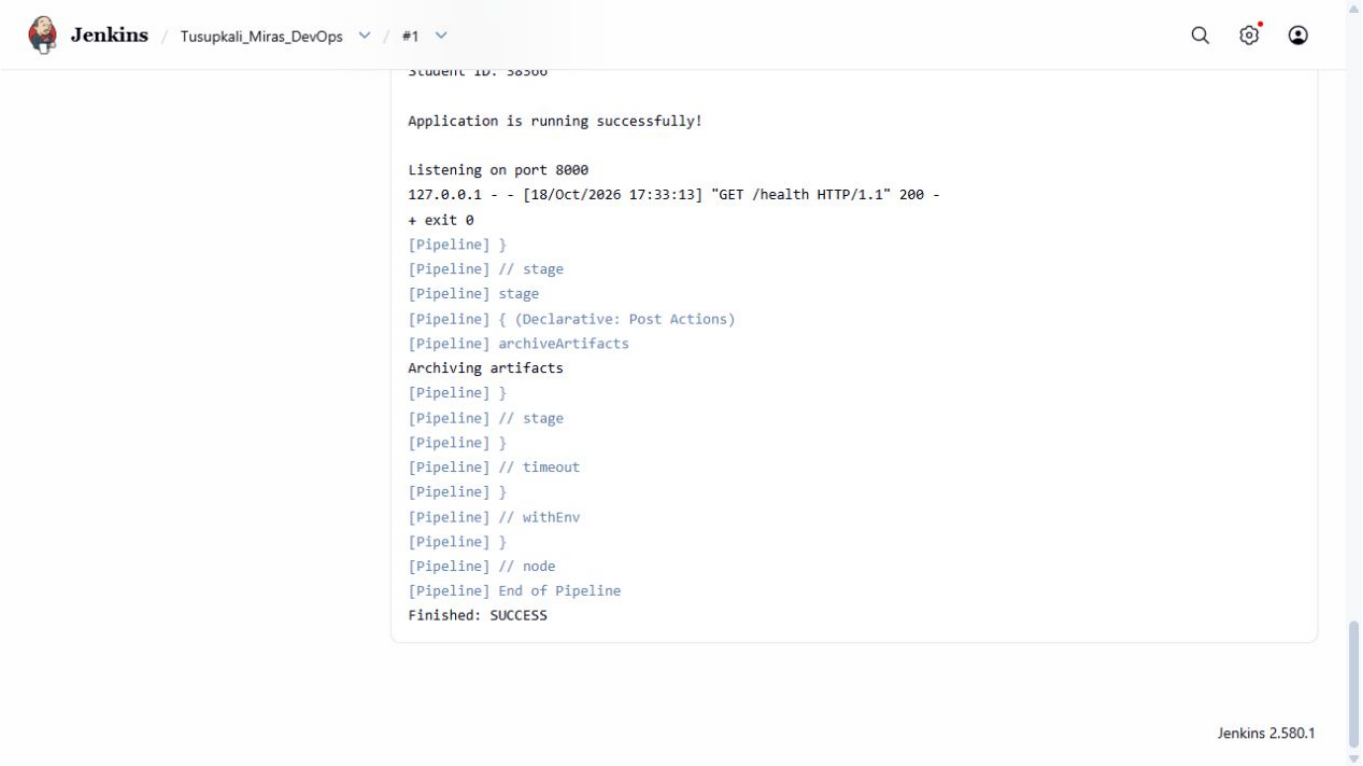


Figure 4. Actual Console Output ending with Finished: SUCCESS.

Selected actual Console Output

```
Student: Miras Tusupkali
Group: IT2-2312
Student ID: 38366
Ran 4 tests in 0.543s
OK
Successfully tagged tusupkali-miras-devops:1
[Pipeline] End of Pipeline
Finished: SUCCESS
```

The application container remained running after the successful Pipeline. Its Docker health check passed, and the application page was opened from the browser. The Jenkins result JSON and complete Console Output were saved as evidence.

6. Integrated workflow and conclusion

Linux -> Git -> GitHub -> Jenkins -> Build -> Test -> Docker Build -> Docker Run -> SUCCESS

Technology	Role in this workflow
Linux	Provides shell tools, permissions, process inspection and the Docker host.
Git	Records meaningful changes, supports branches and merges, and identifies a source revision.
GitHub	Stores the shared source repository from which Jenkins obtains the project.
Jenkins	Runs the version-controlled Pipeline against a specific Git commit.
Docker	Packages the tested application and runs an isolated instance with configurable environment.

When the Pipeline starts, Jenkins obtains Jenkinsfile and checks out the configured branch. Compilation and tests must succeed before the application image is built and deployed. If a stage fails, later ordinary stages are skipped; the post section still attempts to archive the test output. The build is only successful when Jenkins itself records SUCCESS.

Conclusion

The project demonstrates the required Linux operations, personalized Bash script, real Git history and branch merge, public GitHub repository, Docker image and container management, Compose, environment variables and the successful five-stage Jenkins Pipeline. The student must separately record and submit one continuous screen-and-voice video and be able to reproduce and explain the work.

Submission checklist

1. Public GitHub repository link.
2. This PDF report with actual screenshots.
3. Personal continuous video, recommended duration 10-20 minutes (recorded by the student).

Documentation used

Git tutorial: https://git-scm.com/docs/gittutorial

Dockerfile reference: https://docs.docker.com/reference/dockerfile/

Docker Compose: https://docs.docker.com/compose/

Jenkins Pipeline syntax: https://www.jenkins.io/doc/book/pipeline/syntax/

Jenkins configuration as code: https://plugins.jenkins.io/configuration-as-code/

Python unittest: https://docs.python.org/3/library/unittest.html